Presentation Flow (Short Version)

POUND × Flatterie Online Booking System · Al-Vi · Giuz · Kclei · Khalil · about 25 min + Q&A

> **The idea in one line:** explain the problem → show how it's built → **demo one booking from start to finish** → explain what we solved → prove all requirements are met.

1. Who Presents What

#	Who	Talks about	Slides	Time
1	**Al-Vi**	Problem, target client, objectives, scope, user roles	1–5	3–4 min
2	**Giuz**	How it's built: MERN, API, database	6–8	3–4 min
3	**Kclei**	**Demo:** customer account + booking, admin manages users	live	5–6 min
4	**Khalil**	**Demo:** staff confirms → completes a booking, archive	9 + live	6–7 min
5	**Giuz**	Concepts used + problems we solved	10–11	3 min
6	**Al-Vi**	Requirements checklist, future work, conclusion	12–14	2 min
	Everyone	Q&A (answer the topic you presented)		

2. Slides (14 total)

#	Slide	What to put on it
1	Title	Project name, course, members (keep current design)
2	Problem & Client	POUND × Flatterie, SM MOA (sample client). Manual reservations are hard to organize and track.
3	Objectives	Current objectives + "prevent double booking" + "keep records organized with an archive"
4	Scope & Limits	Included: website, booking, 4 modules, 1 branch. Not included: payment, delivery, chat, inventory. Reset email is simulated.
5	User Roles	Customer (book) · Staff (manage bookings, tables) · Admin (everything + user accounts)
6	Architecture	Diagram: React → Express/Node API → MongoDB Atlas
7	API	Main routes (/auth, /users, /tables, /reservations, /customers, /archive) + GET/POST/PUT/PATCH
8	Database	3 collections: users, tables, reservations (a reservation links to a customer and a table)
9	Booking Life Cycle	Pending → Confirmed → Completed (or Cancelled). 2-hour seatings. No double booking.

10	Concepts Used	Login security (hashed passwords, tokens, roles), validation, error messages, responsive design, organized code
11	Problems Solved	Double booking · edit clashing with itself · data lost on restart (→ MongoDB) · faster availability check
12	Requirements ✓	All modules ✓ · **12 users** · **26 completed bookings** · archive ✓ · data cleaned ✓ · client ✓
13	Future Work	Real email/SMS, online payment, more branches, reports
14	Conclusion	Current conclusion + Thank you + GitHub link

3. Hand-off Lines

From → To	Say
Al-Vi → Giuz	*"Those are our users. Giuz will show how we built the system."*
Giuz → Kclei	*"Now let's see it from the user's side. Kclei will start as a customer."*
Kclei → Khalil	*"Our booking is now Pending. Khalil will take it from here as staff."*
Khalil → Giuz	*"Giuz will explain the problems we solved behind these features."*
Giuz → Al-Vi	*"Al-Vi will show how we met every requirement."*

4. Demo Steps

Account	Password
Admin: `giuz.montano@gmail.com`	`Admin@123`
Staff: `rene.baterbonia@gmail.com`	`Staff@123`
Customers (e.g. `maria.santos@gmail.com`)	`Customer@123`

Kclei (customer + admin)

1. Quick look at the home page (menu, map).
2. **Register** a new customer: show an error first (bad email), then succeed.
3. **Book** a table for tomorrow → shows **Pending** + table number.
4. My Reservations → edit **Profile** → **Logout**.
5. **Forgot password** → open the link from the server window → set a new password.
6. Log in as **Paolo** → blocked (account deactivated).
7. As **Admin**: Users page (12 users) → add a staff account → deactivate it.

Khalil (staff + archive)

1. Slide 9: explain the life cycle.
2. As **Rene (staff)**: **Confirm** Kclei's booking.
3. + New Booking, same date/time → the taken table **isn't offered** (no double booking).
4. Yesterday's booking (Syd Villanueva) → **Complete**.
5. Filter **Completed** → 24 shown + 2 in Archive = **26**.
6. **Archive** a completed booking → find it in the Archive page → **Restore**.

> **If the internet fails:** use a phone hotspot. If it still fails, show the backup screenshots.

5. Before the Presentation

- **Fix the slides:** remove "Analytics & Reporting" and "link to full menu" (not in the system); change "Reservation Completed" to "Reservation Submitted (Pending)"; fix typos "o develop", "TARGE", "Benefts", "Conclusiom".
- **Add the missing slides** from section 2 (4–13).
- **Practice once** together, including the hand-off lines.
- **Morning of:** run `npm run seed` (cleans the data), start both servers, test the 3 logins, bring a hotspot.

6. Likely Questions (short answers)

Question	Answer	Who
How do you stop double booking?	Before saving, the system checks if that table is already booked at that date and time. Customers get the smallest free table automatically.	Khalil

Why 2-hour seatings?	Bookings in different seatings never overlap, so the check is simple and reliable.	Khalil
Why archive instead of delete?	The records must be kept. Archived ones are hidden from the main list and can be restored.	Khalil
How are passwords kept safe?	They are hashed (scrambled) with bcrypt, never saved as plain text.	Giuz
How does login work?	The server gives a signed token (JWT). Every request sends it, and the server checks the token and the user's role.	Giuz
Where is the data stored?	MongoDB Atlas (cloud database), in 3 collections: users, tables, reservations.	Giuz
Is the reset email real?	Simulated: the link shows in the server window. It expires in 15 minutes and works once.	Kclei
Can staff manage accounts?	No. Only the admin can add, edit, or deactivate users.	Kclei
How did you make the 20 transactions?	A seed script loads realistic demo data (12 users, 26 completed bookings) and also cleans old test data.	Al-Vi
Why use a real restaurant?	The guideline allows a sample client. The site says it's a student project, not affiliated.	Al-Vi